

Mathematics and Computer Science Research Essay
Fouad Sayegh
June 2024

Title: Multivariate Logistic Regression Model

Research Question: To what extent can a machine learn an algorithm that precisely predicts the probability of someone having a coronavirus based on the use of a multivariate logistic regression model?

Word Count: 3795

Table of Contents

Introduction.....	3
1.0. Linear Regression	4
1.1 Basics of Linear Regression	4
2.0 Logistic Regression Theories.....	5
2.1 Overview of Logistic Regression	5
2.2 Log Odds	5
2.3 Deriving the Equation	6
2.4 Sigmoid Curve (The Logistic Model)	7
2.5 Calculating the slope/coefficient b	8
2.6 Maximum Likelihood.....	9
3.0 Machine Learning Using Python:	11
3.1 Scikit-Learn Library	11
4.0 Coronavirus Application:.....	12
4.1 Random Sampling	12
4.2 Raw Data	12
4.3 Multivariate Logistic Regression	13
4.4 Discrete Data	16
4.5 Accuracy of the Logistic Regression Model	18
5.0 Conclusion	20
6.0 Bibliography:	22
7.0 Appendices:.....	24

Introduction

The global coronavirus epidemic has had a profound effect on the world. Since the coronavirus is highly contagious, people of all ages are continuously paranoid. This ultimately led me to conclude that there must be a mechanism for constructing a model that can predict a person's probability of catching the coronavirus based on a large number of explanatory variables. I've always been interested in programming and machine learning. Numerous courses I've taken that increased my awareness of the various mathematical methods that may be employed in the building of a multivariate logistic regression model allowed me to greatly enhance my abilities. Consequently, I chose to formulate my research question by combining my passion and dedication for computer science with the current status of the world.

Mathematicians have been modeling data for millennia in an attempt to more effectively comprehend the present and prepare for the future. Various techniques have emerged over time to assist with this process, but machine learning has had the greatest impact. As opposed to manually graphing the data, machine learning allows the user to provide a limited amount of data, which is then used to forecasting potential data. Unsupervised machine learning and supervised machine learning are the two parts that make up machine learning. In addition to being fully independent, unsupervised machine learning provides the AI with unclassified data to examine when constructing models from observable patterns. Nevertheless, both components of machine learning ultimately rely on the same mathematical concepts (Burns).

Nave Bayes Classifiers, Multiple Linear Regression, Linear Regression, Support Vector Machines, Logistic Regression, K-Means Clustering, K-Nearest Neighbors, and Decision Trees are fundamental machine learning concepts. To process gathered data and forecast future outcomes, each of the aforementioned concepts is applied. Moreover, they are all developed from mathematical theories. For example, Vector Machines examine data for

regression analysis and classification, while Naive Bayes Classifiers are probability classifiers based on the application of Baye Theorem. However, in supervised learning algorithms, linear and logistic regression models are two frequently used techniques.

1.0. Linear Regression

1.1 Basics of Linear Regression

It's crucial to first realize that linear regression and logistic regression are closely connected. One of the fundamental ideas behind the machine learning method is linear regression, which entails forecasting the value of one variable in relation to another. These variables are often called predictor variables and criterion variables, or responsive variables and explanatory variables (“What Is Linear Regression?” *Statistics Solutions*). The predictor variables are found on the horizontal line, and the criterion variables are found on the vertical line. The data is then gathered and plotted appropriately on the table. The regression line is then shown and fitted to the graph.

$$\text{Regression Question: } Y = a + bx + \epsilon$$

According to the equation above, the dependent variable Y is equal to the y-intercept a plus the coefficient (slope) b multiplied by the independent variable x . If the graph has a lot of points that deviate from the best fit line, some mathematicians choose to include the error ϵ in the dependent variable (Mondal) (Linear Regression).

General linear regression model:

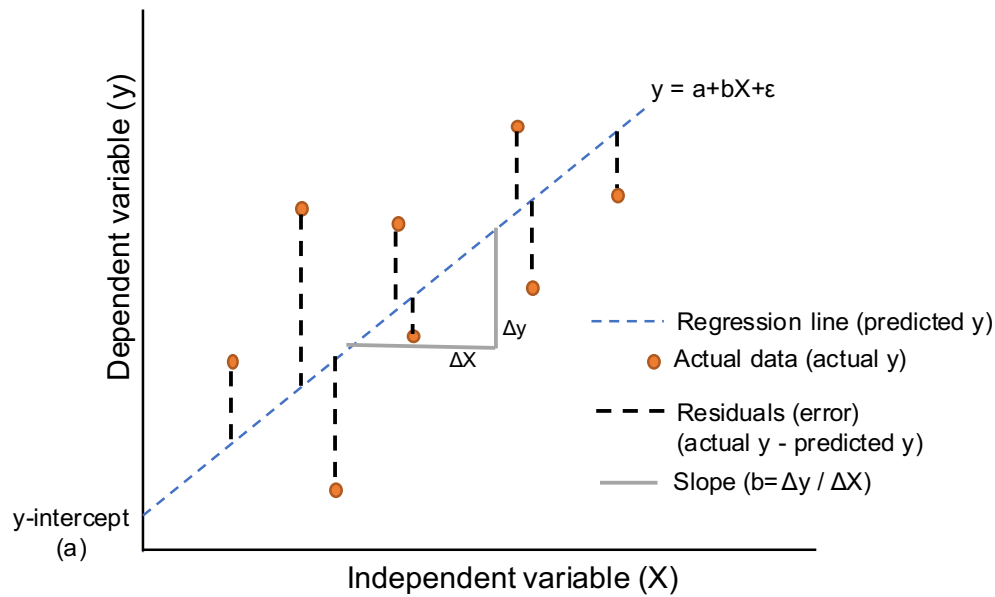


Figure 1 (Bedre)

2.0 Logistic Regression Theories

2.1 Overview of Logistic Regression

Logistic regression and linear regression share many similarities in that they both use correlation to provide an equation to aid in linking variables and forecasting future results. One significant difference is that the equation and line of best fit for linear regression are based on minimizing the sum of the squared errors of the prediction. On the contrary, linear regression is dependent on maximum likelihood and log odds ratios. Prior to exploring these two concepts, it is critical to understand that a binary logistic regression model has a responsive variable that indicates whether the predictions are true or false (1 and 0). This is a factor that linear regression does not take into account (What Is Logistic Regression).

2.2 Log Odds

Odds contrast the number of favorable outcomes with those that are unfavorable, as they are used to calculate the probability that events occur. This formula is represented mathematically as shown below:

$$Odd = \frac{\text{Number of favorable outcomes}}{\text{Number of unfavourable outcomes}}$$

The odds in favor are represented by the formula above. To be able to find the odds against, the reciprocal of the above formula must be found.

Naturally, odds and probabilities have a strong connection. For instance, if an event has a 0.2 probability of happening, its probability of not happening is $1 - 0.2 = 0.8$. Naturally, this results from the knowledge that the probability of an occurrence is always $0 \leq p \leq 1$. Next, we may create the equation shown below:

$$Odds = \frac{p}{1 - p}$$

According to this fundamental equation, the odds of an event happening is equal to the ratio between the chance of an event happening and its non-occurrence. Given that odds are utilized to calculate the regression equation's coefficients, this idea is crucial when discussing logistic regression.

In actuality, the natural logarithm of the odds is obtained after the odds have been calculated. This process, known as linearization, enables us to create the sigmoid function curve by equating the odds to the equation of a line.

$$\ln(odds) = \ln\left(\frac{p}{1 - p}\right)$$

2.3 Deriving the Equation

To explain the remaining theorems and use them to answer the research question, we must first derive the logistic regression equation. Then, we may move on to the next step.

This is firstly done by multiplying the y-intercept plus slop/coefficient by the x value as shown in Section 1.1 and equating it to the $\ln(odds)$ / logit equation aforementioned.

$$\ln\left(\frac{p}{1 - p}\right) = a + bx$$

Rewriting the equation into exponential form:

$$\frac{p}{1-p} = e^{a+bx}$$

Solving for p is the next step, as this is done by multiplying $(1 - p)$ on both sides, resulting in:

$$p = e^{a+bx}(1 - p)$$

Expanding is then done, resulting in:

$$p = e^{a+bx} - e^{a+bx}p$$

All p 's must be on the same side, so the $e^{a+bx}p$ is then changed from the right-hand side to the left-hand side, changing it from negative to positive, and resulting in:

$$p + e^{a+bx}p = e^{a+bx}$$

Factoring out the p and placing it on one side, resulting in:

$$p(1 + e^{a+bx}) = e^{a+bx}$$

The p is then made as the subject of the formula, once more by dividing both sides by $(1 + e^{a+bx})$, resulting in the final estimated regression equation of:

$$p = \frac{e^{a+bx}}{(1 + e^{a+bx})}$$

The equation shown above is in the terms of p (probability), as the x is the variable investigated, a is the y-intercept of the log odds function, and b is the coefficient/slope of the log odds function.

2.4 Sigmoid Curve (The Logistic Model)

The sigmoid curve, which depends on the sigmoid function, is the name given to the graph of a logistic regression model. The similarity of the equations helps show their correlation, with the sigmoid function having the following:

$$S(x) = \frac{1}{1 + e^{-x}}$$

As opposed to the logistic regression, which has the following equation derived in 2.3:

$$p = \frac{e^{a+bx}}{(1 + e^{a+bx})}$$

A sigmoid function's graph appears as follows:

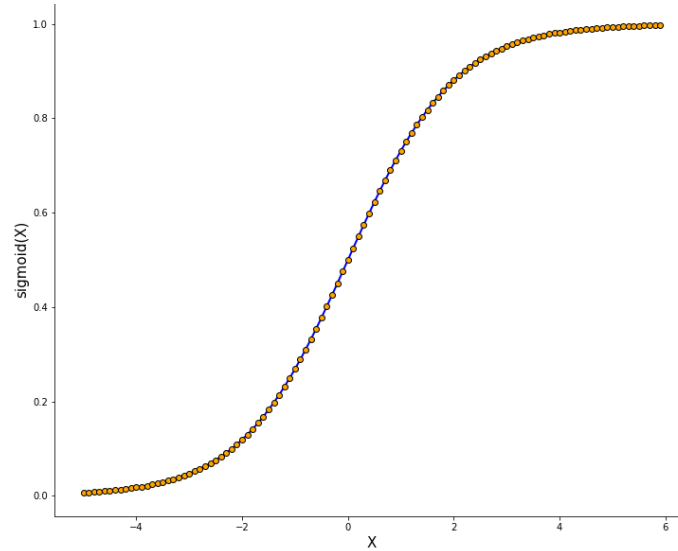


Figure 2 (Thorn)

It is evident that the sigmoid function's range, which corresponds to the range of probabilities, is between 0 and 1. We can also observe that the inflexion point is at 0.5 in Figure 2. This is crucial, as an example on such is if a logistic regression were to be used to assess if a person had a fatal condition, the probability of 0.58 would be meaningless and we would instead just look at the inflexion point, where anything below it would be 0 and anything above would be 1.

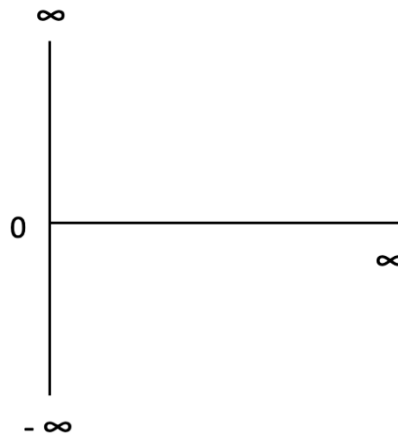
2.5 Calculating the slope/coefficient b

A mixture of the previously described concepts can be used to complete the difficult process of determining the coefficient b in the logistic regression equation. Using logit is the simplest method to accomplish this. The regression equation may be written as follows:

$$p = \frac{e^{\text{logit}}}{(1 + e^{\text{logit}})}$$

Thus, we may dispense with the need of two separate variables for b (coefficient/slope) and a (y-intercept) and instead depend on the log(odds) graph. The concept at hand is extremely vague since it necessitates a lot of trial and error.

A plain axis is firstly drawn which has a domain of $[0, \infty)$ and a range of $(-\infty, \infty)$. This is how it would appear:



Following the preparation of the graph, a line of best fit is drawn; no calculations are necessary for this step, and the line is drawn entirely at random. However, the line must be reasonable, given the survey's hypothesis and purpose. After a best-fit line has been created, the study's data's x values are projected onto it to provide the appropriate y values. Since these y values are equal to the logit, the resulting equation is as follows:

$$p = \frac{e^y}{(1 + e^y)}$$

The sigmoid function that results from plotting all the p values is then plotted. Values may be predicted in accordance with this sigmoid function, which is now the logistic regression model, however the accuracy is quite low.

2.6 Maximum Likelihood

Trial and error are the basis for the method described in this section, however there are steps that may be taken to increase accuracy. Utilizing maximum likelihoods is the most common

approach. Plotting p on the sigmoid function after calculating it for each of the values under consideration is crucial to begin with. The graph must then be divided by the mean, which is typically 0.5 and the sigmoid curve's inflexion point. This allows us to classify anything as true or false. After classifying the values as true or false, the true values are multiplied together, and the false values are then individually deducted from 1 and are multiplied with both the true values and each other. We are attempting to measure the maximum likelihood; thus, the false values are removed by 1, which implies that we are trying to determine the likelihood that each value is true as opposed to false. We conclude with the subsequent equations:

False Values (left / under the point of inflexion on the sigmoid function):

$$likelihood = (1 - probability\ of\ false)$$

Overall Maximum Likelihood:

$$Maximum\ Likelihood = (probability\ of\ true) \times (1 - probability\ of\ false)$$

All of the values are multiplied by one another after the aforementioned method is performed for each one.

$$Maximum\ Likelihood = (p_1 \times (1 - f_1)) \times (p_2 \times (1 - f_2)) \dots \times (p_n \times (1 - f_n))$$

Where n is the number of values, p is the probability to be true, and f is the probability to be false.

However, statisticians prefer to calculate the $\log(likelihood)$ instead, therefore, the equation above will slightly alter to:

$$\log(likelihood) = \log(p_1) + \log(1 - f_1) \dots + \log(p_n) + \log(1 - f_n)$$

In this instance, n is the number of values, p is the probability, and f is the probability under the mean. As one can realize that the additions have been changed to multiplications.

After calculating the $\log(likelihood)$ or probability, we return to the log graph in Section 2.6, rotate the line, reposition the values along that best-fit rotated line, and then recalculate the probability using the same equation:

$$p = \frac{e^{\text{logit}}}{(1 + e^{\text{logit}})}$$

After discovering the probabilities, we once more categorize them as true or false, as previously explained, and repeat the procedures for calculating the log(likelihood). The values are then compared; if it is closer to zero, the line must be rotated in the same direction but with a less/flatter steep gradient, and if the value after rotation is more than zero, the line must be rotated in the other direction. The logistic regression model must be refined until it is accurate enough to produce reliable predictions, which occurs when the sigmoid curve matches the data as closely as feasible. Naturally, rotating the line to get a number almost equal to zero is a time-consuming procedure; here is where computer programming and machine learning algorithms come into play.

3.0 Machine Learning Using Python:

Numerous programming languages can be utilized to execute the majority of logistic regression data modeling activities. Python, in particular, is a language in which I have a significant amount of knowledge and proficiency. Due to its simple syntax and adaptability, it is regarded as one of the most effective programming languages ever devised.

3.1 Scikit-Learn Library

Major programming languages, such as Python, act as a foundation for other programmers to build additional applications (libraries) that assist in expanding the possible usage of the aforementioned language. Scikit-Learn for Python is a machine learning package that lays a major emphasis on regression and clustering. The Scikit-Learn package allows me to perform Section 2.6 exceptionally well. In addition, the algorithm is able to continuously minimize the error on each trial as opposed to relying on trial and error, hence reducing the trial-and-error procedure to almost no time at all.

4.0 Coronavirus Application:

4.1 Random Sampling

Coronavirus is still a young virus, and although it has been a global problem for nearly three years, its behavior is still largely unpredictable. The majority of data collection, however, has relied on random sampling, in which a group of people selected at random represents the whole population. To accomplish this, I obtained data from 30 individuals who tested positive for coronavirus. I will compare the trends I found to information published by the “World Health Organization” and then proceed to develop my own logistic regression model if the patterns are equal.

4.2 Raw Data

As can be seen in the table to the right, the only variable I first utilized to determine whether a case was positive or not was the number of symptoms the individual had out of a possible total of five. The primary symptoms of the coronavirus, according to the Centers for Disease Control and Prevention (CDC), include a dry cough, loss of taste, loss of smell, overall fatigue, and fever.

Positive Case	Number of Symptoms
1	3
2	2
3	1
4	1
5	4
6	2
7	1
8	1
9	2
10	2
11	3
12	4
13	1
14	3
15	4
16	2
17	5
18	5
19	2
20	3
21	1
22	1
23	1
24	4
25	2

Figure 3: Raw Data 1

4.3 Multivariate Logistic Regression

When many independent variables have an impact on the regression model and the probabilities that result, this is known as multivariate logistic regression. Now that we've extended the table of raw data in Figure 6, we may see the table shown in Figure 4:

The distance from another positive case has been introduced as a new variable. Although I can't expect the individual to remember the exact measurement, they had three choices: 10 meters, around 6 meters, and less than 2 meters. This offers a variety of possibilities and makes it possible to monitor patterns more readily. A logistic regression model must be created in order to fully comprehend this data. We must first recall the regression equation:

Positive Case	Number of Symptoms	Distance From Positive Case
1	3	6
2	2	2
3	1	2
4	1	1
5	4	2
6	2	6
7	1	6
8	1	6
9	2	10
10	2	2
11	3	2
12	4	6
13	1	10
14	3	10
15	4	6
16	2	1
17	5	1
18	5	1
19	2	2
20	3	6
21	1	2
22	1	2
23	1	10
24	4	10
25	2	6

Figure 4: Raw Data 2

$$p = \frac{e^{a+bx}}{(1 + e^{a+bx})}$$

This equation must be changed to comply with more variables, as the one above is an equation for only 1 variable. This is done with the following equation:

$$p = \frac{e^{a+bx+b_2x_2...b_nx_n}}{(1 + e^{a+bx+b_2x_2...b_nx_n})}$$

The x_n represents the variables in the equation above.

As shown in Section 2.6, finding the coefficient is a fairly complicated task that may be considerably sped up by using programming. The Python library Scikit-Learn may be used to discover coefficients, as mentioned in Section 3. Extra information should be collected in advance from individuals who exhibit symptoms but test negative and organize it in binary form as given in the table shown in Figure 5.

Positive/Negative	Number of Symptoms	Distance From Positive Case
1	3	6
1	2	2
1	1	2
1	1	1
1	4	2
1	2	6
1	1	6
1	1	6
1	2	10
1	2	2
1	3	2
1	4	6
1	1	10
1	3	10
1	4	6
1	2	1
1	5	1
1	5	1
1	2	2
1	3	6
1	1	2
1	1	2
1	1	10
1	4	10
1	2	6
0	1	10
0	2	10
0	1	10
0	1	10
0	1	10
0	1	10
0	1	6
0	1	2
0	2	2
0	1	2
0	1	2
0	1	1
0	1	10
0	1	10
0	2	10

Figure 5: Raw Data 3

The data is gathered using my code in the Appendices and includes:

$$a = 1.5931$$

$$b_1 = 0.3782$$

$$b_2 = -0.3185$$

When we enter the values into the initial equation, we obtain:

$$p = \frac{e^{1.5931+0.3782x_1-0.3185x_2}}{(1 + e^{1.5931+0.3782x_1-0.3185x_2})}$$

Now that we have this equation, we can forecast things correctly. The probability that someone with 3 symptoms (x_1) will become infected with the coronavirus if they are within 2 meters (x_2) of someone with the virus is:

$$p = \frac{e^{1.5931+0.3782(3)-0.3185(2)}}{(1 + e^{1.5931+0.3782(3)-0.3185(2)})}$$

$$p = 0.8899959765$$

They are therefore highly prone to get infected with the virus.

A second scenario, with the same symptoms but being 8 meters distant from a positive case, results in:

$$p = \frac{e^{1.5931+0.3782(3)-0.3185(8)}}{(1 + e^{1.5931+0.3782(3)-0.3185(8)})}$$

$$p = 0.5448044956$$

Since he was farther away from the positive case than the last individual, the result indicates that the subject is unlikely to test positive. This has naturally shown out to be correct since maintaining a healthy social distance is one of the key techniques to avoid contracting the virus.

4.4 Discrete Data

To accurately forecast the risk of someone catching the coronavirus for the last parts of the essay, we need to include a few different variables. The discrete variable, which in this case refers to whether or not the person is wearing a face mask, is a non-continuous variable. The responses may be expressed in binary form as 0 for no and 1 for yes since it can only be no or yes. As a result, we include a third column to represent that variable shown in Figure 6 below.

Positive/Negative	Number of Symptoms	Distance From Positive Case	Face Mask
1	3	6	0
1	2	2	1
1	1	2	1
1	1	1	0
1	4	2	1
1	2	6	0
1	1	6	0
1	1	6	0
1	2	10	0
1	2	2	0
1	3	2	1
1	4	6	0
1	1	10	0
1	3	10	1
1	4	6	1
1	2	1	1
1	5	1	1
1	5	1	1
1	2	2	0
1	3	6	0
1	1	2	1
1	1	2	1
1	1	10	1
1	4	10	0
1	2	6	0
0	1	10	0
0	2	10	1
0	1	10	1
0	1	10	1
0	1	10	1
0	1	10	0
0	1	6	0
0	1	2	1
0	2	2	1
0	1	2	1

0	1	2	1
0	1	1	0
0	1	10	0
0	1	10	1
0	2	10	0

Figure 6: Raw Data 4

The following variables are obtained by plugging the given table into the Scikit-learn algorithm:

$$a = 1.9281$$

$$b_1 = 0.5608$$

$$b_2 = -0.3345$$

$$b_3 = -1.3821$$

Plugging in the variables into the equation:

$$p = \frac{e^{1.9281+0.5608x_1-0.3345x_2-1.3821x_3}}{(1 + e^{1.9281+0.5608x_1-0.3345x_2-1.3821x_3})}$$

The same data in Section 4.3 is used to test the equation:

$$p = \frac{e^{1.9281+0.5608(3)-0.3345(2)-1.3821(0)}}{(1 + e^{1.9281+0.5608(3)-0.3345(2)-1.3821(0)})}$$

$$p = 0.9498602137$$

This forecasts the probability that someone standing within 2 meters (x_2) of a positive case showing 3 symptoms (x_2) and wearing no mask (x_3). There is a 94.99% (rounded to 4 decimal places) risk that they will catch the infection. Now, assuming the same individual is wearing a mask:

$$p = \frac{e^{1.9281+0.5608(3)-0.3345(2)-1.3821(1)}}{(1 + e^{1.9281+0.5608(3)-0.3345(2)-1.3821(1)})}$$

$$p = 0.82626724$$

This demonstrates that using a mask reduces the risk of contracting the virus by over 12.36% (rounded to 4 decimal places).

Last but not least, let's take the second person into account:

$$p = \frac{e^{1.9281+0.5608(3)-0.3345(8)-1.3821(0)}}{(1 + e^{1.9281+0.5608(3)-0.3345(8)-1.3821(0)})}$$

$$p = 0.7179873455$$

As this is without a mask, at a distance of # meters, and displaying # symptoms.

With a mask:

$$p = \frac{e^{1.9281+0.5608(3)-0.3345(8)-1.3821(1)}}{(1 + e^{1.9281+0.5608(3)-0.3345(8)-1.3821(1)})}$$

$$p = 0.3899315388$$

Once more, the possibilities have decreased by over 32.81% (rounded to 4 decimal places) as a result of them donning masks.

4.5 Accuracy of the Logistic Regression Model

The aforementioned logistic regression model may now forecast an individual's likelihood of developing the coronavirus depending on a range of other variables. But it happens frequently that a PCR test, for instance, will provide a false negative or false positive result. The accuracy of my logistic regression model needed to be calculated in order for my application to remain as realistic as feasible. I had to go outside the bounds of my mathematic curriculum once more. After much investigation, I discovered the AUC of the ROC curve as a means of gauging accuracy.

This method is extensively employed in medicine, particularly when dealing with tests like the PCR test where it is common to have both false negative and false positive readings. The area under the receiver operating characteristic (ROC) curve, a curve used to assess binary classification models like the one constructed, must be found using this technique. Specificity and sensitivity are the two most crucial factors that affect the curve. The difference between the two is subtle; specificity evaluates the percentage of patients who were correctly identified as being in the negative category, while sensitivity evaluates the percentage of coronavirus patients who were accurately identified.

The following equations condense the aforementioned information:

$$\text{Sensitivity (True Positive Rate)} = \frac{\text{True Positive (TP)}}{\text{False Negative (FN)} + \text{True Positive (TP)}}$$

$$\text{Specificity (True Negative Rate)} = \frac{\text{True Negative (TN)}}{\text{False Positive (FP)} + \text{True Negative (TN)}}$$

We also need to define the false positive rate, as the ROC shows the true positive rate vs. the false positive rate:

$$\text{False Positive Rate (FPR)} = \frac{\text{False Positive (FP)}}{\text{False Positive (FP)} + \text{True Negative (TN)}}$$

This may be stated even more succinctly as:

$$\text{FPR} = 1 - \text{Specificity}$$

We now put the FPR on the horizontal axis and the TPR on the vertical axis to create the ROC curve. The range and domain are both 0 to 1 inclusive because we are working with probability (Bhandari).

The threshold, or the point at which the model decides whether something is positive or negative, true or false, is an important consideration. This point in my model is 50% or 0.5. Although the model can show the likelihood of having the coronavirus, it is more practical to categorize the patient as positive or negative; in this situation, if the probability is less than 50%, they are negative, and if it is more than 50%, they are positive.

Using both the ROC curve and the distribution of the pertinent probabilities, the following illustration depicts the ideal situation (when the model is 100 percent accurate):

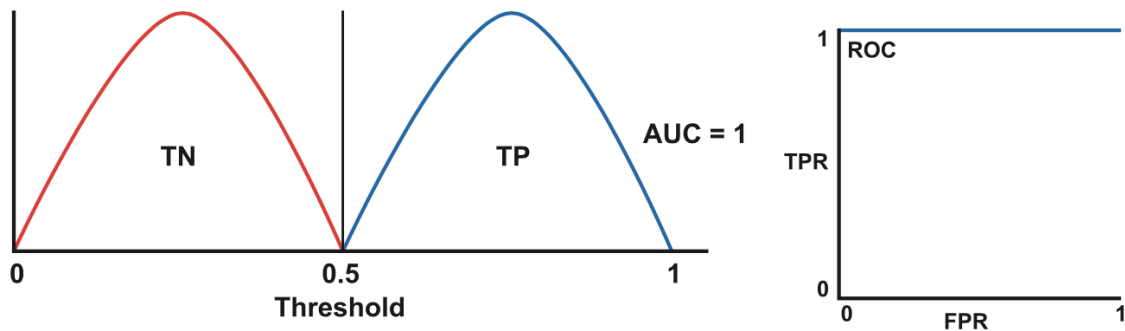


Figure 7: Symbolizes an Ideal Situation for The Model

Since there is no overlap at all between the distributions in this instance, the ROC curve's area under the curve will be 1 (the whole area). An example of a situation containing false positives and negatives is shown below:

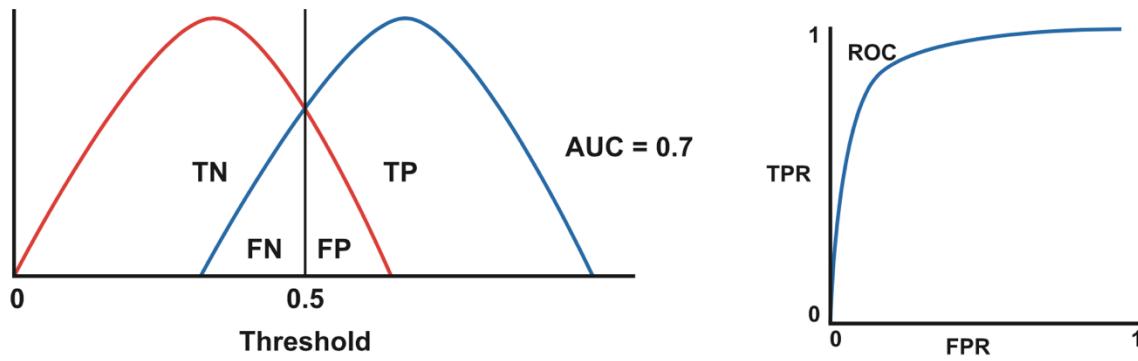


Figure 8: Symbolizes Inaccuracies in The Model

Here, we observe that the overlapping distributions result in false positives and negatives, lowering the AUC from 1 to 0.7. This is just simply an example and does not accurately demonstrate my logistic regression model.

As a result of this trend, we may draw certain conclusions. The model is unable to differentiate between a person who has the coronavirus and a person who does not if the AUC = 0.5. The model is less accurate when the AUC is closer to 0, and more accurate when it is closer to 1. AUC = 0, which denotes that all positives are classified as negatives by the model, is the worst-case scenario.

5.0 Conclusion

With the scant data offered, the developed logistic regression model has proven to perform remarkably well. Additionally, I employed computer programming (specifically, Python) to streamline the procedure. The advantages of using both continuous and discrete data, particularly in the context of my research question, became clear to me as a result of my investigation.

To begin, I used two parameters: positive cases represented by 1s (positive) and 0s (negative), and the number of symptoms ranging from 1 to 5. I then added the third distance parameter from a positive case, which were represented through 1, 2, 6, and 10 meters. Finally, I added the fourth parameter of a face mask represented by 1's (wearing a face mask) and 0's (not wearing a face mask) (not wearing a face mask). I then used my code to calculate the probability of how the probability varies with different parameters, which proved that my code was correct.

There are still a few modifications that could be made to the model to improve it even further. First, there is the addition of new variables, such as the period of exposure and whether the positive condition was encountered inside or outside. Furthermore, utilizing more than 40 samples will increase the model's overall accuracy. This is crucial since the standard deviation, mean, etc. will become more accurate as more data is acquired and more trends are observed.

My essay research gave me a fantastic opportunity to practice problem-solving and critical thinking techniques while encountering a variety of difficult and challenging concepts. Additionally, I also had the chance to learn more about the fundamentals of machine learning, which I will undoubtedly be able to apply when I enroll in my computer science undergraduate program.

Finally, returning to the research question, a machine learning method based on a dichotomous logistic regression model can indeed accurately estimate the likelihood that a person would get the coronavirus. Determining the parameters for a multivariate logistic function is difficult mathematically, thus utilizing Python programming makes the task much easier. Not only can this forecast the probability of having coronavirus based on the four factors mentioned above, but it can also be tweaked to be applied in other circumstances with additional parameters in the future.

6.0 Bibliography:

- Bedre, Renesh. *General Linear Regression Model*. Online Image, “Calculating residuals in regression analysis [Manually and with codes],” *Data science blog*, 18 Nov. 2022, <https://www.reneshbedre.com/blog/learn-to-calculate-residuals-regression.html>.
- Bhandari, Aniruddha. “Guide to AUC-Roc Curve in Machine Learning : What Is Specificity?” *Analytics Vidhya*, 10 Jan. 2023, 15 Jan. 2023, <https://www.analyticsvidhya.com/blog/2020/06/auc-roc-curve-machine-learning/>.
- Burns, Ed. “What Is Machine Learning and Why Is It Important?” *Enterprise AI*, TechTarget, 30 Mar. 2021, 30 Sep. 2022, <https://www.techtarget.com/searchenterpriseai/definition/machine-learning-ML>.
- “Linear Regression: Simple Steps, Video. Find Equation, Coefficient, Slope.” *Statistics How To*, 20 Nov. 2021, 8 Dec. 2022, <https://www.statisticshowto.com/probability-and-statistics/regression-analysis/find-a-linear-regression-equation/>.
- Mondal, Sourav. “Linear vs Logistic Regression: Linear and Logistic Regression.” *Analytics Vidhya*, 30 Nov. 2022, 20 Dec. 2022, <https://www.analyticsvidhya.com/blog/2020/12/beginners-take-how-logistic-regression-is-related-to-linear-regression/#:~:text=The%20Differences%20between%20Linear%20Regression,Logistic%20regression%20provides%20discrete%20output>.
- Thorn, James. *Logistic Regression*. Online Image, “Logistic Regression Explained,” *Towards Data Science*, Towards Data Science, 8 Feb. 2020, 25 Nov. 2022, <https://towardsdatascience.com/logistic-regression-explained-9ee73cede081>.
- “What Is Linear Regression?” *Statistics Solutions*, 10 Aug. 2021, 2 Jan. 2023, <https://www.statisticssolutions.com/free-resources/directory-of-statistical-analyses/what-is-linear-regression/>.

“What Is Logistic Regression?” *IBM*, 2 Dec. 2022, <https://www.ibm.com/topics/logistic-regression>.

7.0 Appendices:

Appendix 1:

```
 # Importing the Necassary Libraries
from sklearn import metrics
from sklearn.metrics import classification_report
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
import pandas as pd
from sklearn.metrics import roc_curve
from matplotlib import pyplot
from sklearn.metrics import roc_auc_score
```

Appendix 2:

```
[ ] info = pd.read_csv("CoronaVirusData.csv") # Reading the Data csv File
display(info) # Displaying the DataFrame to see what it looks like

print(info['Final_Result'].value_counts()) # This returns the number count of positive and negative Final Results
features = info.drop('Final_Result', axis=1) # Dropping the column column I want to test for the algorithm used to predict accurate results
result = info['Final_Result'] # saving the Final_Result Column to test if my Algorithm was able to accurately predict the outcomes.

print(features.columns) # writes the columns without the Final_Result column to make sure that my DataFrame is ready to test

xTrain, xTest, yTrain, yTest =sklearn.model_selection.train_test_split(features, result, test_size = 0.2, random_state = 5)

print("X-Train: \t{}\nX-Test: \t{}\nY-Train: \t{}\nY-Test: \t{}".format(xTrain,xTest,yTrain,yTest))
# The above print statement writes the number of rows the algorithm trains and tests itself on.
# Training will be 80% of the Data, while testing will be 20% of the other unseen data.

# Giving the data for the model to perform its calculations to predict the results
represent = LogisticRegression(random_state=5, multi_class="auto", penalty="elasticnet", solver="lbfgs").fit(xTrain, yTrain)
prediction = represent.predict (xTest)

# writing down the model parameters
parameters = represent.get_params()
print(parameters)
print('Intercept: \t{}\nCoefficients: \t{}'.format(represent.intercept_,represent.coef_))
```

Appendix 3:

```
 # Writing down the results to compare them

NSprobability = [0] * len(yTest)
LSpability = represent.predict(xTest)[: ,1]
NS_Accuracy = roc_auc_score(yTest, NSprobability)
LR_Accuracy = roc_auc_score(yTest, LSpability)
print("ROC AUC={}".format(lr_auc))
NS_False_Positive_Rate, NS_True_Positive_Rate, dummy_Variable1 = roc_curve(yTest, NSprobability)
LR_False_Positive_Rate, LR_True_Positive_Rate, dummy_Variable2 = roc_curve(yTest, LSpability)

# Plotting the graph to show the results
plt.plot(LR_False_Positive_Rate, LR_True_Positive_Rate, marker="--", label="Logistic")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.legend()
plt.show()
```